



/-

ClockStandard.lean — A⁵O Clock Standard (A5O-CLK-1.0) revisited in Lean 4.

Purpose: separate the arithmetic content of the January 2026 disclosure ("A⁵O Clock Standard 17.5 ns / 57.142857 MHz", "57 million checks per core") from its physical content, and prove exactly the arithmetic part.

All quantities are integers in fixed units so that no real-number library is needed and every theorem is decidable by the kernel:

time : picoseconds (ps)

frequency : the pair (num, den) meaning num/den hertz

Toolchain: Lean 4.33.1, core library only, no Mathlib, no axioms.

-/

namespace A5O.Clock

/-! ## 1. The clock as stated -/

/-- Stated period: 17.5 ns = 17 500 ps. -/

def periodPs : Nat := 17500

/-- Stated frequency: 400/7 MHz, i.e. $(400 \cdot 10^6) / 7$ Hz. -/

def freqNum : Nat := 400 * 10 ^ 6

def freqDen : Nat := 7

/-- One second in picoseconds. -/

def secondPs : Nat := 10 ^ 12

/-! ## 2. The period and the frequency are the same statement.

$f \cdot T = 1 \Leftrightarrow (400 \cdot 10^6 / 7 \text{ Hz}) \cdot (17\,500 \text{ ps}) = 10^{12} \text{ ps}$

$\Leftrightarrow 400 \cdot 10^6 \cdot 17\,500 = 7 \cdot 10^{12}$ -/

theorem period_matches_frequency : freqNum * periodPs = freqDen * secondPs := by decide

/-! ## 3. Ticks per second is a division, not a measurement.

$10^{12} \text{ ps} / 17\,500 \text{ ps} = 57\,142\,857 \text{ remainder } 2\,500.$ -/

def ticksPerSecond : Nat := secondPs / periodPs

theorem ticksPerSecond_eq : ticksPerSecond = 57142857 := by decide

theorem ticksPerSecond_remainder : secondPs % periodPs = 2500 := by decide

/-! ## 4. "57.142857 MHz" is a six-digit truncation of $400/7$, not an exact value.

$$57\,142\,857 / 10^6 < 400/7 < 57\,142\,858 / 10^6 \quad -/$$

theorem stated_value_is_truncation :

$57142857 * \text{freqDen} < \text{freqNum} * 1 \wedge \text{freqNum} * 1 < 57142858 * \text{freqDen}$:= by decide

theorem stated_value_not_exact : $57142857 * \text{freqDen} \neq \text{freqNum}$:= by decide

/-- The repeating block: $10^6 \equiv 1 \pmod{7}$, so the decimal expansion of $400/7$ repeats with period 6 (the block 142857). -/

theorem decimal_period_six : $10^6 \% 7 = 1$:= by decide

/-! ## 5. Where "57 million checks per core" lives.

A gate that completes k decisions per tick performs $k \cdot \text{ticksPerSecond}$ decisions per second. The number 57 142 857 follows from $k = 1$ and the chosen period. The claim that $k = 1$ holds on any physical core is a HYPOTHESIS about hardware and software; Lean records it, it does not discharge it. -/

/-- A gate characterised only by how many decisions it completes per clock tick. -/

structure Gate where

checksPerTick : Nat

def checksPerSecond (g : Gate) : Nat := g.checksPerTick * ticksPerSecond

/-- The physical claim, stated as a hypothesis on a gate. -/

def OneCheckPerTick (g : Gate) : Prop := g.checksPerTick = 1

/-- Under the hypothesis, the headline number is forced by arithmetic alone. -/

theorem fifty_seven_million (g : Gate) (h : OneCheckPerTick g) :

checksPerSecond g = 57142857 := by

unfold checksPerSecond OneCheckPerTick at *

rw [h, ticksPerSecond_eq]

/-- Without the hypothesis the headline number is not implied by the clock:

a gate completing zero decisions per tick has the same clock and zero throughput. -/

theorem clock_alone_forces_nothing :

```

    ∃ g : Gate, checksPerSecond g = 0 := ⟨⟨0⟩, by decide⟩

```

```

/-- "Checks per core" is "checks per second" only under one further
    hypothesis: one core runs exactly one gate at this clock. That is also
    physical; it is recorded here as a definition, not proved. -/

```

```

def checksPerCore (g : Gate) (gatesPerCore : Nat) : Nat :=
  gatesPerCore * checksPerSecond g

```

```

theorem checksPerCore_one (g : Gate) (h : OneCheckPerTick g) :
  checksPerCore g 1 = 57142857 := by
  unfold checksPerCore; rw [fifty_seven_million g h]

```

```

end A5O.Clock

```

```

/-! ## Audit -/

```

```

#print axioms A5O.Clock.period_matches_frequency
#print axioms A5O.Clock.ticksPerSecond_eq
#print axioms A5O.Clock.stated_value_is_truncation
#print axioms A5O.Clock.stated_value_not_exact
#print axioms A5O.Clock.fifty_seven_million
#print axioms A5O.Clock.clock_alone_forces_nothing
#print axioms A5O.Clock.checksPerCore_one

```